

Comparison of Regular and Context-Free Languages

Objectives:

- Automata Theory Overview
- Prepared for: Review of Regular and Context-Free Languages (Rules-defined)
- Focus: Regular Expressions, DFAs, and CFGs

By: MBPacot
CSC 112 (1st Sem., A.Y. 2025-2026)

Formal Languages in Automata

- A *formal language* is a set of strings formed from an alphabet.
- Each language type defines how strings can be generated or recognized.
- Two major types: Regular Languages and Context-Free Languages.

Regular Languages

- Defined by *Regular Expressions (RE)*.
- Recognized by *Finite Automata (DFA/NFA)*.
- Can describe simple patterns of symbols.
- Do not require memory of previous symbols.

Regular Expression Example

- RE: $(a+b)^*b$
- Language: All strings over $\{a,b\}$ that end with b.
- Accepted by a DFA or NFA (automaton) – Show DFA - Try!
- Example strings: b, ab, aab, bab, bb.

Define the regular expression for the language:

$L = \{\omega \mid \omega \text{ contains the substring } 001\}$.

Then, give three (3) sample strings that belong to this language.

Answer:

$L = \{\omega \mid \omega \text{ contains the substring } 001\}$.

Then, give three (3) sample strings that belong to this language.

RE = $(0+1)^*001(0+1)^*$

TRY!

Set of strings consisting of an even number of a's followed by an odd number of b's. Then, give three (3) sample strings that belong to this language.

**Define first the L*

Answer:

Set of strings consisting of an even number of a's followed by an odd number of b's. Then, give three (3) sample strings that belong to this language.

$$L = \{a^m b^n \mid m \text{ is even and } n \text{ is odd}\}$$

$$RE = (aa)^* b (bb)^*$$

Define the regular expression for the language:

$L = \{a^p \mid p \text{ is prime}\}$

Then, give three (3) sample strings that belong to this language.

Answer:



Pumping Lemma for Regular Languages

For showing that certain languages are not regular (By Contradiction)

Let L be a regular language. Then there exists a constant p (pumping length) such that for every string w in L such that $|w| \geq p$, we can break w into three strings, $w = xyz$ such that:

1. $|y| > 0$
2. $|xy| \leq p$
3. For all $k \geq 0$, the string xy^kz is also in L

Why we care for RL?

Regular languages can be recognized by finite automata, which are extremely simple computational models.

Step 1: Language

$$L(G) = \{ab^n \mid n > 0\}$$

- Strings start with **a** and have **at least one b**.
- Examples: **ab**, **abb**, **abbb**, **abbbb**, ...

Step 2: Pumping length

Let the pumping length $p = 3$.

Step 3: Pick a string $s \in L(G)$

- Choose $s = abbbbb$ (1 **a** + 5 **b's**)
- Check length: $|s| = 6 \geq p = 3$ ✓

Step 4: Split $s = xyz$

- Conditions: $|xy| \leq p = 3, |y| > 0$
- The first $p = 3$ symbols of **s** are: **abb**
- So possible splits:
 1. $x = a, y = b, z = bbbb$ ✓ valid choice
 2. $x = ab, y = b, z = bbb$ ✓ another valid choice

Let's pick **first option**:

$$x = a, \quad y = b, \quad z = bbbb$$

Step 5: Pump y

Check xy^iz for different i :

i	xy^iz	String
0	$xz = a bbbb$	abbbb ✓ in $L(G)$
1	$xyz = a b bbbb$	abbbbb ✓ original string
2	$xy^2z = a b b bbbb$	abbbbbbb ✓ in $L(G)$
3	$xy^3z = a b b b bbbb$	abbbbbbbb ✓ in $L(G)$

Remarks

- All pumped strings **still start with** **a** **and have ≥ 1** **b**, so they are in $L(G)$.

Applying Pumping Lemma = L $\neq$ RL

RE: $(a+b)^*b$

Language: All strings over $\{a,b\}$ that end with b.

Accepted by a DFA or NFA (automaton)

Example strings: b, ab, aab, bab, bb.

Let $P = 3$:

Let L be a regular language. Then there exists a constant p (pumping length) such that for every string w in L such that $|w| \geq p$, we can break w into three strings, $w = xyz$ such that:

1. $|y| > 0$
2. $|xy| \leq p$
3. For all $k \geq 0$, the string xy^kz is also in L

Try! - Individual Task

Prove to be a RL using pumping lemma (show proof) and give equivalent automaton if it is a regular language.

1) $L = \{a^m b^n \mid m \text{ is even and } n \text{ is odd}\}$

Context-Free Languages

- Defined by *Context-Free Grammars (CFG)*.
- Can describe nested or recursive structures.
- Require a stack for memory.

CONTEXT FREE GRAMMAR V.0.0

ScriptC Programming Language

Token Definitions (symbols and characters classes)

<variable_name> → a|b|..z|A|B|..Z
<integers> → [-]?[0-9]+
<decimals> → [-]?[0-9]+\.[0-9]+
<string_value> → \"([^\"]|\\\")*\">
<character_value> → \'.*\'

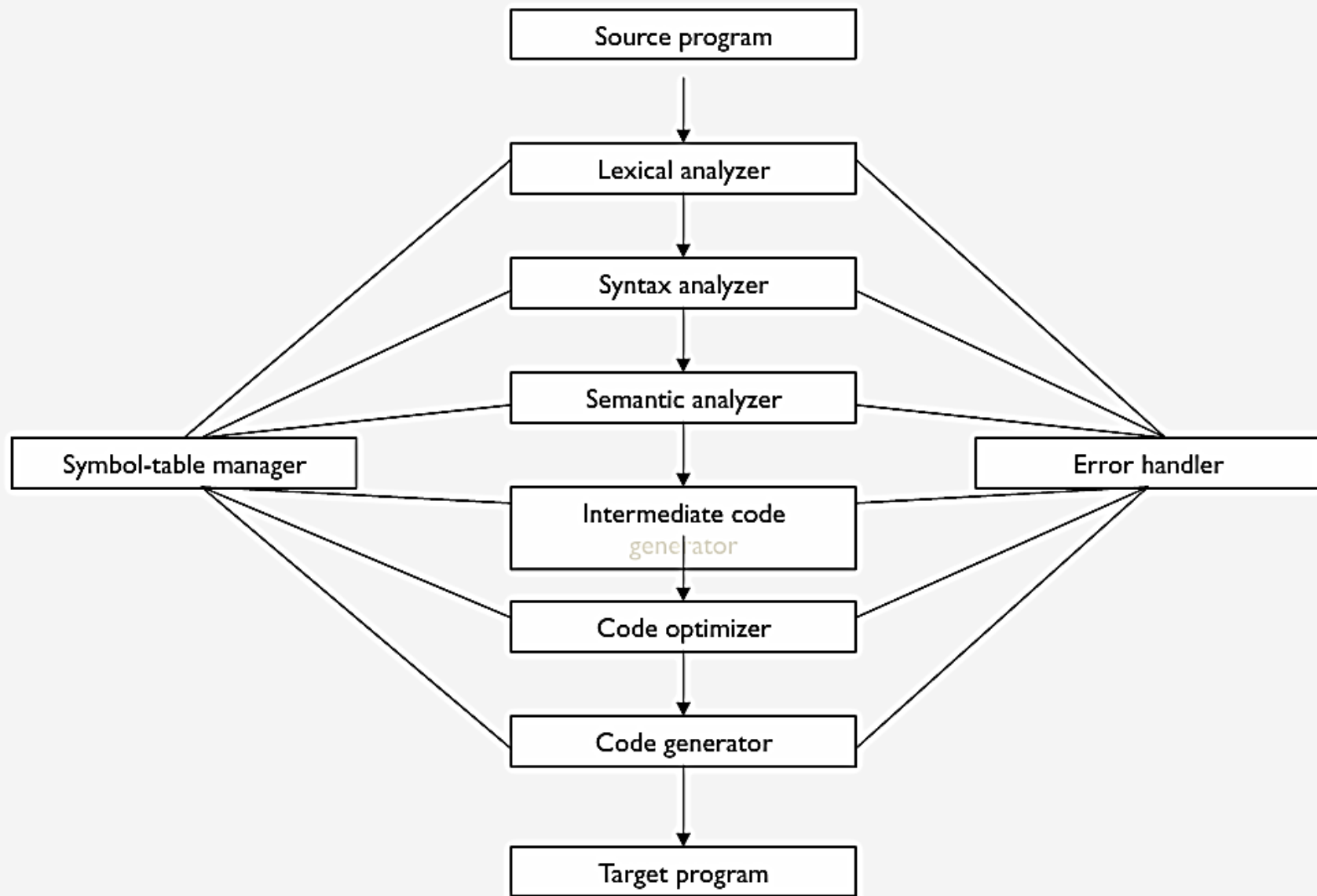
CFG Productions (declaration and initialization)

<T>

<numbers_statements> → <variable_name> ':' <datatype>
| <variable_name> '=' <expression>
| <variable_name> '=' <variable_name>
| <variable_name> ':' <datatype> '=' <expression>
| <variable_name> ':' <datatype> '=' <variable_name>

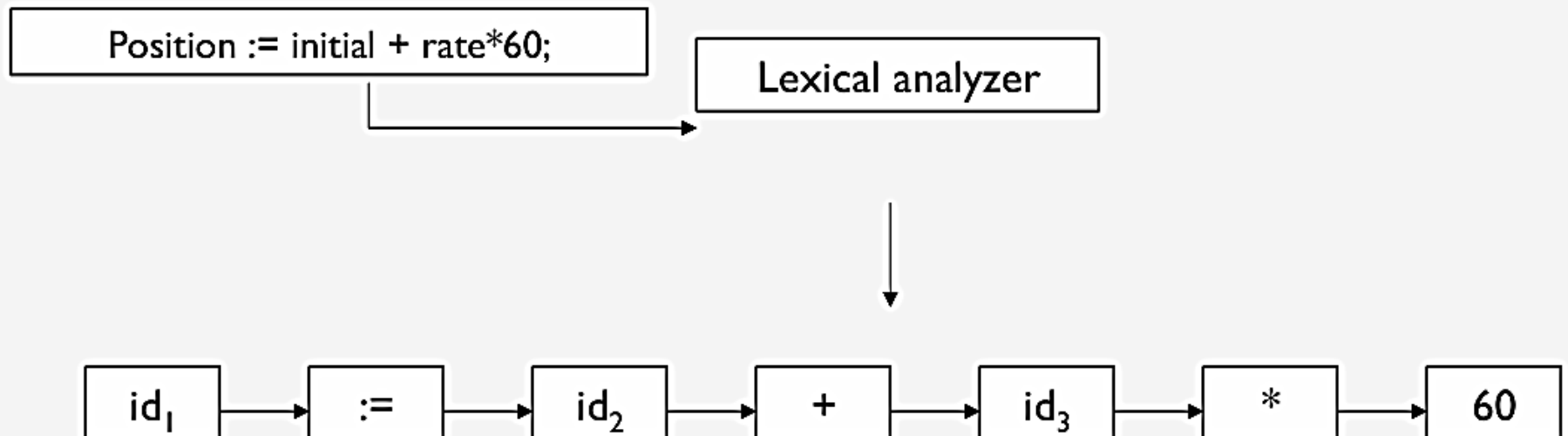
<NT>

OVERVIEW



LEXICAL ANALYSIS

- Converts the stream of input characters into a stream of tokens that becomes the input to the following phase (syntax analysis)



OVERVIEW

- Syntax
 - what the program looks like
 - CFG (Context-free grammars) or BNF (Backus-Naur Form)
- Semantics – what the program means

CONTEXT-FREE GRAMMAR

- A set of **nonterminals**
- A set of tokens (**terminal** symbols)
- A set of productions
 - nonterminal (*left side*)
 - arrow
 - sequence of tokens and/or nonterminals (*right side*)
- A designated **start** symbol

$$G = (V, T, P, S)$$

CONTEXT-FREE GRAMMAR

Conventions

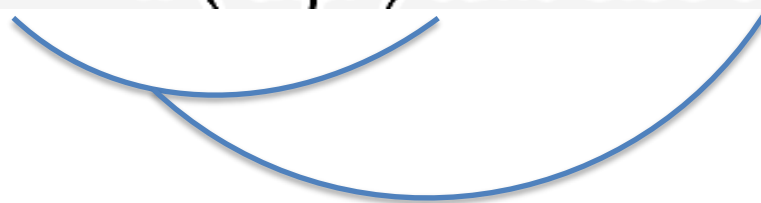
- Productions for the start symbol are listed first
- Digits, signs ($\leq$) and **boldface** strings (**while**) are terminals(tokens)
- Nonterminals are *italicized*
- Any nonitalicized name or symbol may be assumed to be a token
- Productions with the same nonterminal on the left can have their right sides grouped
- The string containing zero tokens is written as ϵ (**empty** string)

**A context-free grammar can be left-recursive.*

CONTEXT-FREE GRAMMAR

if (expression) statement **else** statement

stmt → **if** (*expr*) *stmt* **else** *stmt*



CONTEXT-FREE GRAMMAR

list $\rightarrow$ *list* + *digit*
list $\rightarrow$ *list* - *digit*
list $\rightarrow$ *digit*
digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

} EXPANDED FORM

list $\rightarrow$ *list* + *digit* | *list* - *digit* | *digit*
digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

} COMPACT FORM

$L(G) = \{d((+|-)d)^* \mid d \in \{0 \text{ to } 9\}\}$

RE = $[0-9]([+ -] [0-9])^*$

CONTEXT-FREE GRAMMAR

CFG:

$$S \rightarrow aB$$
$$B \rightarrow b$$

Language: {"ab"}

This grammar is **not recursive** — it produces only **one specific pattern** (no repetition, no nesting).

CONTEXT FREE GRAMMAR V.0.0

ScriptC Programming Language

Token Definitions (symbols and characters classes)

<variable_name> → a|b|..z|A|B|..Z
<integers> → [-]?[0-9]+
<decimals> → [-]?[0-9]+\.[0-9]+
<string_value> → \"([^\"]|\\\")*\"
<character_value> → \'.*\'

**Remember this
(FINAL PROJECT)?**

CFG Productions (declaration and initialization)

<T>

<numbers_statements> → <variable_name> ':' <datatype>
| <variable_name> '=' <expression>
| <variable_name> '=' <variable_name>
| <variable_name> ':' <datatype> '=' <expression>
| <variable_name> ':' <datatype> '=' <variable_name>

<NT>

CFG Derivation

We derive using a CFG to show how a string belongs to the language that the grammar defines.

Derivation = proof (step-by-step) that a string is generated by the grammar.

So, if you have a grammar G and a string w , performing a derivation shows that $w \in L(G)$.

DERIVATION

- Derive strings
 - Begin with the start symbol
 - Repeatedly replace a nonterminal with the right side of the production for that nonterminal
- Derived strings form the *language* defined by the grammar

DERIVATION (LEFT-MOST DERIVATION)

list → *list* + *digit*

list → *list* - *digit*

list → *digit*

digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

list + *digit*

list - *digit* + *digit*

digit - *digit* + *digit*

9 - *digit* + *digit*

9 - 5 + *digit*

9 - 5 + 2

9 - 5 + 2

$L(G) = \{d((+|-)d)^* \mid d \in \{0 \text{ to } 9\}\}$

RE = $[0-9]([+ -] [0-9])^*$

CFG - Productions:

$S \rightarrow T B$

$B \rightarrow + T B \mid - T B \mid + T \mid - T$

$T \rightarrow \text{digit}$

$\text{digit} \rightarrow 0 \mid 1 \mid \dots \mid 9$

Derive $3 + 4 - 2$: (left-most derivation)

Step 1: $T B$

Step 2: $\text{digit } B$

Step 3: $3 + T B$

Step 4: $3 + \text{digit } B$

Step 5: $3 + 4 B$

Step 6: $3 + 4 - T$

Step 7: $3 + 4 - \text{digit}$

Final Step: $3 + 4 - 2$

$L(G) = \{d((+|-)d)^+ \mid d \in \{0 \text{ to } 9\}\}$

$RE = [0-9]([+ -] [0-9])^+$

**Enforce to have one operator*

Context-Free Grammar Example

- CFG: $S \rightarrow aSb \mid \varepsilon$
- **Language:** $\{a^n b^n \mid n \geq 0\}$
- Example derivations:
- $S \Rightarrow \varepsilon$, $S \Rightarrow aSb \Rightarrow ab$, $S \Rightarrow a aSb b \Rightarrow aabb$ *and so on.*
- This language cannot be expressed with Regular Expressions.

Try! Individual Task

$$L = \{a^m b^n \mid m \text{ is even and } n \text{ is odd}\}$$

Construct the CFG and prove that 'aaaabb' is a string that belongs to $L(G)$.

CFG - Solution:

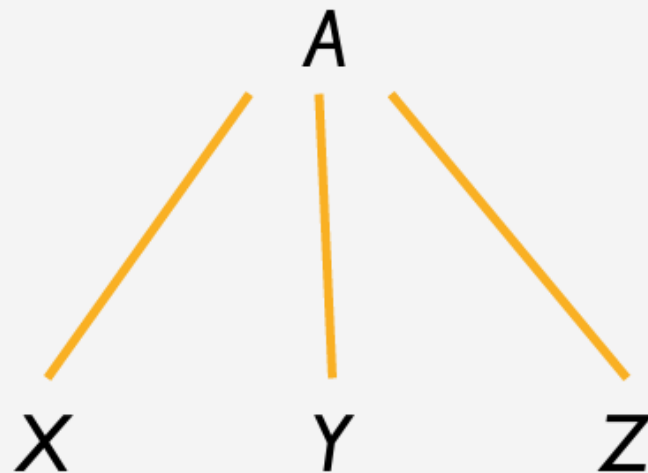


CFG - **Solution** : Cont.



PARSE TREES

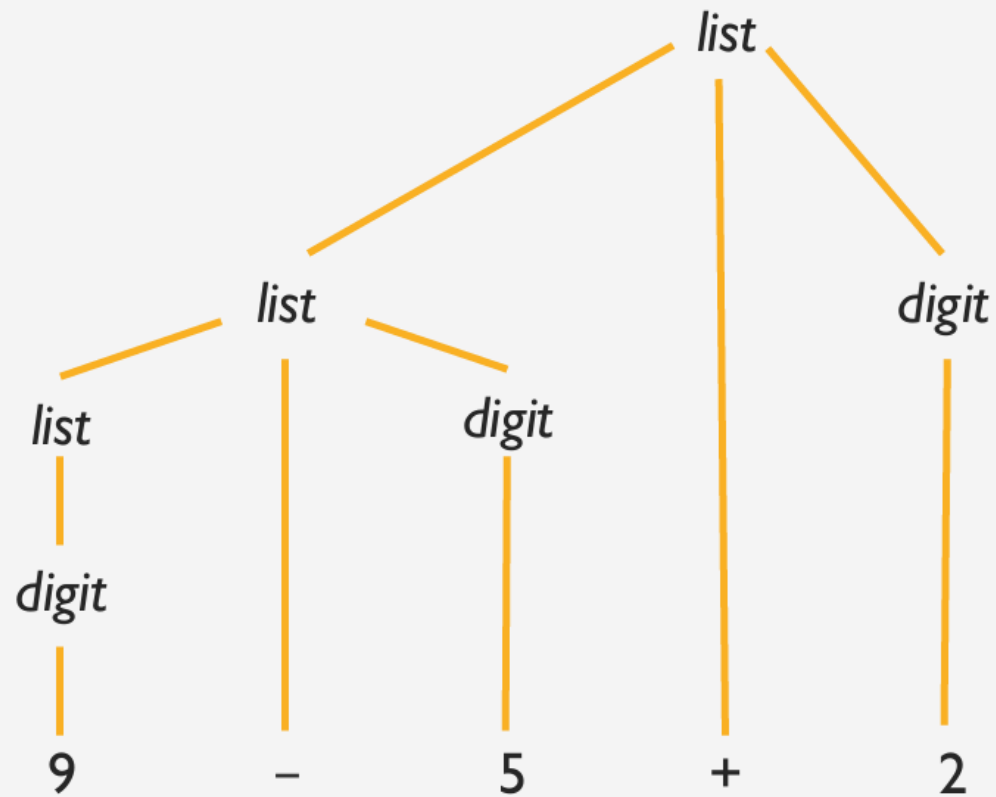
$A \rightarrow XYZ$



PARSE TREES

- The root is labeled by the start symbol
- Each leaf is labeled by a terminal (token) or ϵ
- Each interior node is labeled by a nonterminal
- If A is the nonterminal labeling some interior node and the labels of the children from left to right are $X_1, X_2, \dots, X_n$, then $A \rightarrow X_1 X_2 \dots X_n$ is a production

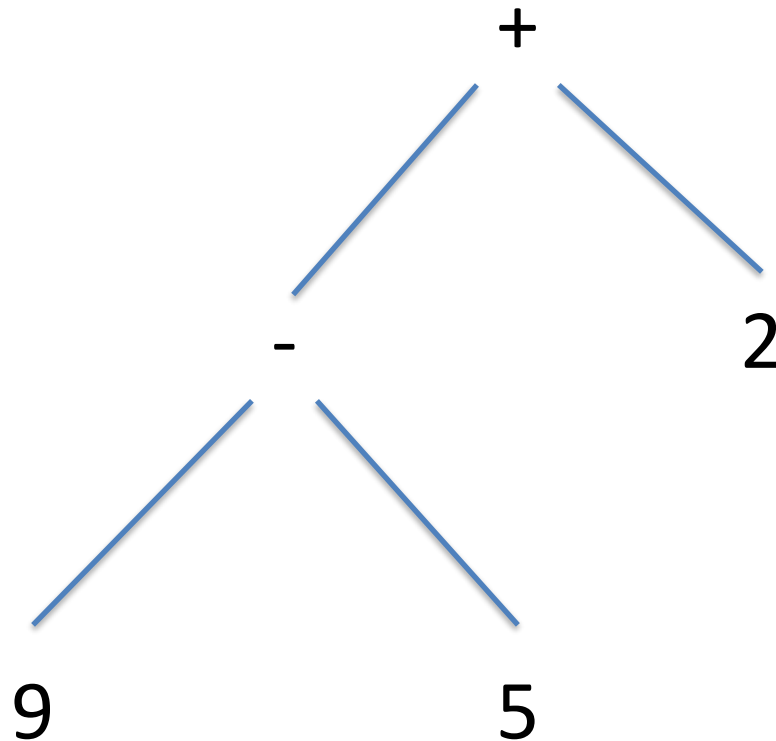
DERIVATION



← **yield** of the tree

AST

String: 9 - 5 + 2



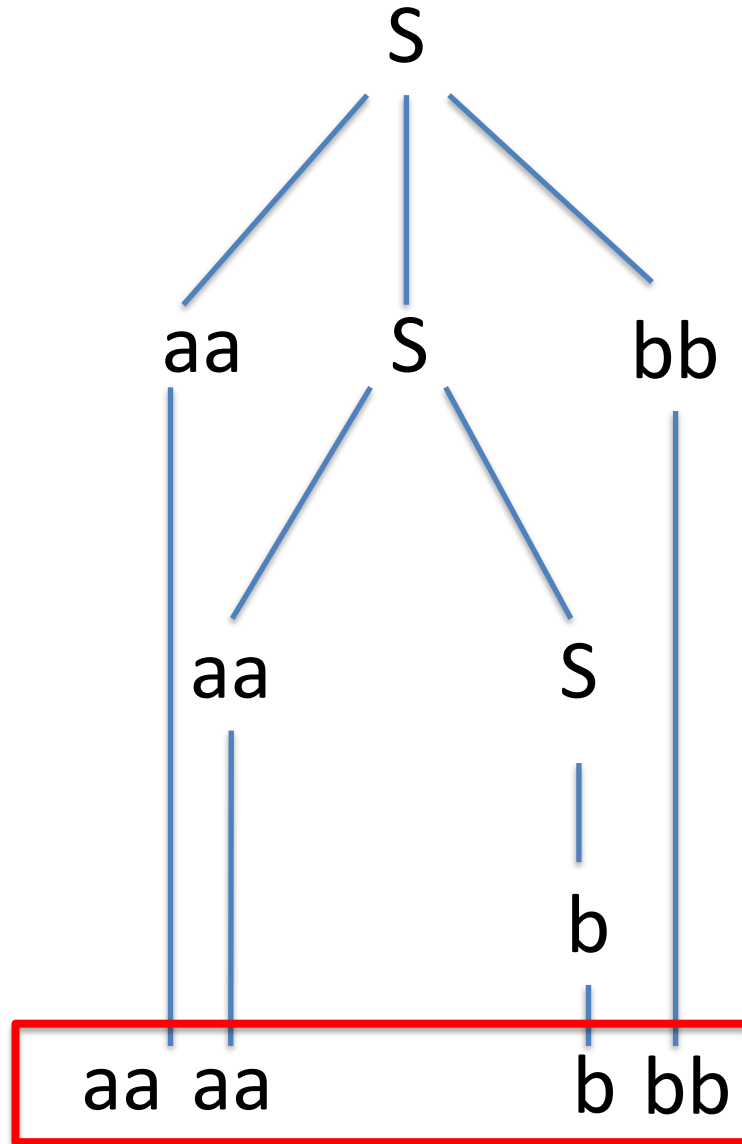
Try! Individual Task

Construct the equivalent Parse Tree from this given example CFG production:

$$s \rightarrow aaS \mid aaSbb \mid aab \mid b$$

Derived string “aaaabbbb”:

String: aaaabbbb



AMBIGUITY

(any give expression/s)

- A grammar can have more than one parse tree generating a given string of tokens

$$G = \langle \{string\}, \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, P, string \rangle$$

with the production:

$$string \rightarrow string + string \mid string - string \mid 0 \mid 1 \mid \dots \mid 9 \quad \leftarrow \text{ambiguous}$$

$$9 - 5 + 2$$

This grammar produced ambiguity since the parser could apply either operator first, depending on how the expression is parsed.

Example: $2 + 3 * 4 = ?$

CFG: $\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{number}$

$\text{expr} * \text{expr}$

$\text{expr} + \text{expr} * \text{expr}$

//apply either operator first

$\text{number} + \text{expr} * \text{expr}$

$2 + \text{expr} * \text{expr}$

$2 + \text{number} * \text{expr}$

$2 + 3 * \text{expr}$

$2 + 3 * \text{number}$

$2 + 3 * 4$

*This grammar **doesn't separate** addition and multiplication at different **(levels of precedence)**. Both the $+$ and $*$ operators are at the same level, which means that the parser could apply either operator first, depending on how the expression is parsed.*

Example: $2 + 3 * 4$

CFG (fixed):

$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$

$\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$

$\text{factor} \rightarrow \text{number} \mid (\text{expr})$

$\text{expr} + \text{term}$

$\text{term} + \text{term}$

$\text{factor} + \text{term}$

$\text{number} + \text{term}$

$2 + \text{term}$

$2 + \text{term} * \text{factor}$

$2 + \text{factor} * \text{factor}$

$2 + \text{number} * \text{factor}$

$2 + 3 * \text{number}$

$2 + 3 * 4$

**FIXED version on
ambiguity!**

*//apply the * first as higher
precedence*

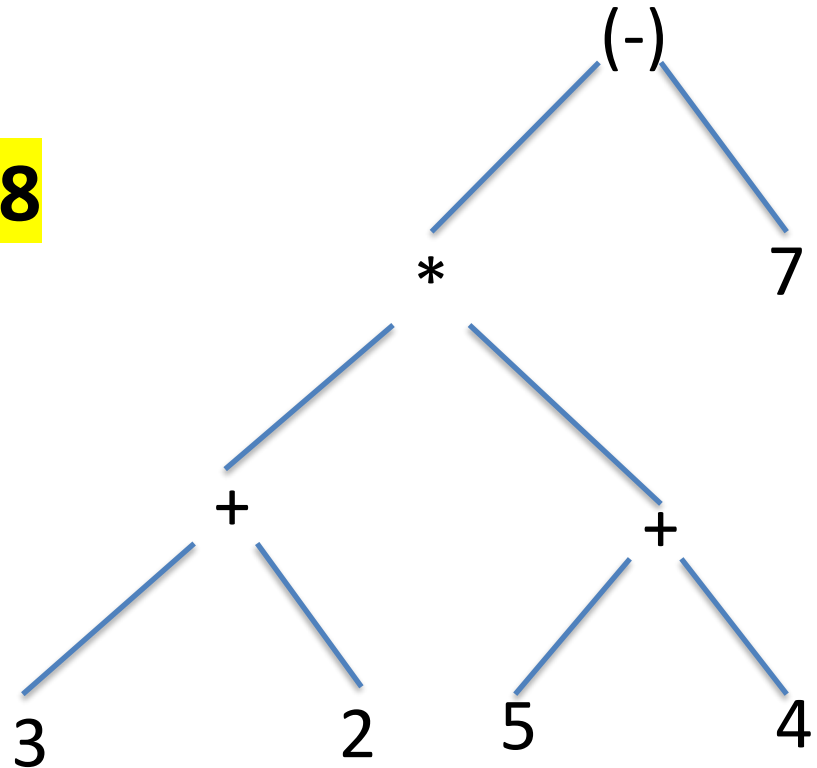
Order of Precedence for Common Operators

- 1) Parentheses $()$: Anything inside parentheses is evaluated first, ensuring that these operations are prioritized over others.
- 2) Exponents $^$ (or $**$ in some languages) : Exponentiation (raising to a power) is usually evaluated next after parentheses.
- 3) Multiplication $*$ and Division $/$: These operators have the same level of precedence and are evaluated from left to right (left-associative).
- 4) Addition $+$ and Subtraction $-$: These operators have the same precedence as multiplication and division, and they are evaluated last in the absence of parentheses (left-associative – if TIE).

Evaluate Expressions

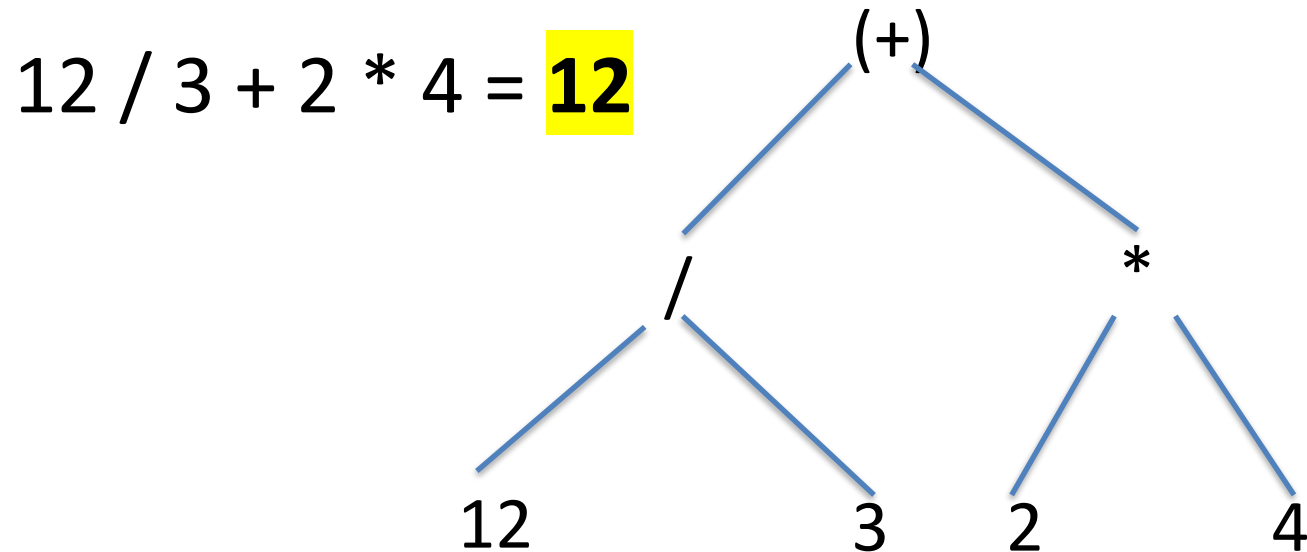
- Determine the correct answer of an expression together with its equivalent AST.

$$(3 + 2) * (5 + 4) - 7 = \mathbf{38}$$



Evaluate Expressions

- Determine the correct answer of an expression together with its equivalent AST.



Try! Individual Task

1. $10 / 2 * 3$ (Answer and AST)

2. $8 + 4 * 2$ (Answer and AST)

Example Comparison

- Regular: $(a+b)^*b \rightarrow$ All strings ending in b.
- Context-Free: $a^n b^n \rightarrow$ Equal number of a's and b's.
- Regular $\rightarrow$ simple pattern recognition.
- Context-Free $\rightarrow$ hierarchical structure handling.

Summary and Key Takeaways

- Regular Languages $\leftrightarrow$ REs, DFAs.
- Context-Free Languages $\leftrightarrow$ CFGs, Pumping Lemma (contradiction).
- Every regular language is context-free, but not vice versa.
- Understanding both helps design compilers and syntax analyzers.

END

Try! Quiz - Individual Task

